

Introduction to Garbage Collection

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

Garbage Collection

The runtimes of many programming languages feature **dynamic memory management**.

The first aspect of that management is **allocation**, that is, the program can create **objects** whose lifetime can be that of the program itself. In particular, they can outlive the procedure or function that created them, contrary to **local variables**, which live in the **call stack**.

Those objects exist in a memory area called the **heap**. More precisely, a **continuous block** is allocated for one object, whose size depends on the **type** of the object.

Garbage Collection

For example, in Java, we can declare a type for the cells in a linked list of integers as follows:

```
class Cell {  
    int    content;  
    Cell  next;  
}
```

Each cell contains an integer and a **pointer** to another cell. The pointer is implicit in the recursive occurrence of the type name `Cell`. It is either the address of an object in memory, or a special value **nil** (noted `null` in Java). Consequently, each cell requires at least two **words** (one for each field and perhaps more, depending on the data layout).

Garbage Collection

A cell can be allocated with the built-in function `new`, like so:

```
Cell c = new Cell ();
```

A word is the smallest amount of memory that can be addressed and filled by a primitive type or a pointer, that is, 64 bits in modern computers. In other words, we can imagine memory as a table with addresses as keys and words as values, which are interpreted either as an address (pointer) or not.

Then we can modify the contents of the fields it contains, as many times as we wish to. Every call to `new` returns the **address of a new block** drawn from the available memory.

Garbage Collection

Because each allocation removes from available, finite memory, it is necessary to **free the memory used by useless objects**, which we need to define and determine what are those objects.

In some programming languages, like C, this task is entirely delegated to the programmers. They have a predefined procedure `free` that frees a memory block:

```
free(c);
```

After it is executed, the block pointed at by `c` is considered available again, so its address can be reused in a future allocation. Two common errors are the use a freed object and to forget to free an object.

Garbage Collection

In Java, there is no equivalent to the `free` procedure of C because the identification and freeing of useless objects is left to the runtime system.

That part of the runtime is called the **garbage collector** (GC). At regular intervals, it interrupts the program, analyses the heap, find out the useless objects and free them.

This technique was invented by McCarthy, creator of the functional programming language LISP in 1960. It is widely used in mainstream languages like Java, C#, Haskell or OCaml.

Beyond avoiding the two errors above, a GC can free many objects at a time, which is more efficient than manually.

How does a GC determine what objects are useless?

Garbage Collection

